

WOW64 兼容性检测与 32/64 位 PEB 差异（知识点整理）

● 涉及的核心结构体与类型

pNtQueryInformationProcess（函数指针类型定义）

```
typedef NTSTATUS(NTAPI * pNtQueryInformationProcess)(
    HANDLE,           // 目标进程句柄
    PROCESSINFOCLASS, // 要查询什么信息
    PVOID,            // 输出缓冲区地址
    ULONG,            // 缓冲区大小
    PULONG             // 实际写入字节数（可 NULL）
);
```

- 为什么需要自定义函数指针类型：NtQueryInformationProcess 不在 kernel32.lib 或标准 C++ 库中，无法直接链接调用。必须通过 GetProcAddress 动态获取其地址，再强转为该函数指针类型才能调用。

PROCESSINFOCLASS（枚举类型）

```
typedef enum _PROCESSINFOCLASS {
    ProcessBasicInformation = 0,
    ProcessWow64Information = 26,    // 本课使用的参数
    ProcessImageFileName = 27,
    // ... 更多枚举值
} PROCESSINFOCLASS;
```

- ProcessWow64Information 值为 26：查询目标进程的 WOW64 状态。

ULONG_PTR（自适应指针类型）

```
ULONG_PTR wow64Info = 0;
```

- 在 32 位编译环境下，ULONG_PTR 为 4 字节（unsigned int）
- 在 64 位编译环境下，ULONG_PTR 为 8 字节（unsigned __int64）
- 用它接收 NtQueryInformationProcess 的返回结果，可以自适应位数，避免在 32/64 位环境下长度不匹配。

NTSTATUS（原生 API 返回类型）

```
NTSTATUS status = NtQuery(hProcess, ...);
```

- NTSTATUS 是 LONG 的别名（4 字节有符号整数）
- status == 0（即 STATUS_SUCCESS）表示成功
- 非 0 值表示错误（如 0xC0000005 表示访问违规）

● 涉及的关键宏与常量

ProcessWow64Information（值为 26）

```
NtQuery(hProcess, ProcessWow64Information, &wow64Info, sizeof(wow64Info), NULL);
```

作用：告诉 NtQueryInformationProcess，我们要查询的是“目标进程是否为 WOW64”。

返回值规律：

- wow64Info == 0 → 目标进程为纯 64 位
- wow64Info != 0 → 目标进程为 32 位(WOW64)，且该值就是该进程的 64 位 PEB 地址

NTAPI（调用约定宏）

```
typedef NTSTATUS(NTAPI pNtQueryInformationProcess)(...);
```

- 展开为 __stdcall：Windows 原生 API 统一使用 __stdcall 调用约定（参数从右向左入栈，

被调用者清理栈)。

➤ 为何必加：如果漏写 NTAPI 编译器默认用__cdecl，调用时栈会失衡导致程序崩溃。

STATUS_SUCCESS (值为 0)

```
if (status != 0) { /* 失败 */ }
```

➤ 原生 API 的标准成功返回码。

➤ 等价于 status == 0。

● **完整的用链（内存寻址与调用链路图）**

第 1 步：获取 ntdll.dll 基址

```
GetModuleHandleA("ntdll.dll")
```

└── 返回 HMODULE (模块基址)，失败返回 NULL

第 2 步：获取 NtQueryInformationProcess 函数地址

```
GetProcAddress(hNtdll, "NtQueryInformationProcess")
```

└── 返回 FARPROC (函数地址)，失败返回 NULL

└── 强转为 pNtQueryInformationProcess 类型

第 3 步：调用函数查询目标进程位数

```
NtQuery(hProcess, ProcessWow64Information, &wow64Info, sizeof(wow64Info), NULL)
```

└── status == 0 → 调用成功

└── 读取 wow64Info 的值

第 4 步：判断结果

```
wow64Info == 0 → 纯 64 位进程
wow64Info != 0 → 32 位进程(WOW64)
                且值 = 64 位 PEB 地址
```

第 5 步：关闭句柄（清理资源）

```
CloseHandle(hProcess)
```

总结：

➤ GetModuleHandleA 拿到 ntdll.dll 基址（这个 DLL 是原生 API 的“仓库”）。

➤ GetProcAddress 从 ntdll.dll 导出表中找到 NtQueryInformationProcess 函数地址。

➤ 强转类型后调用，传入 ProcessWow64Information 参数。

➤ 检查 status 是否为 0，若成功则读取 wow64Info 判断位数。

➤ 清理 hProcess 句柄。

● **核心偏移量速查表**

目标内容	64 位 PEB 偏移	32 位 PEB 偏移
Ldr (PEB 内)	PEB + 0x18	PEB + 0x0C
InMemoryOrderModuleList (Ldr 内)	Ldr + 0x20	Ldr + 0x14
InMemoryOrderLinks (LDR_DATA_TABLE_ENTRY 内)	+0x10	+0x08
DllBase	+0x30	+0x18

EntryPoint	+0x38	+0x1C
SizeOfImage	+0x40	+0x20
FullDllName	+0x48	+0x24
BaseDllName	+0x58	+0x2C

关键记忆口诀：32 位偏移大约是 64 位偏移的一半（指针大小 4 vs 8）。但并非严格等比，需以实际结构体字段对齐为准。

● 其他必要的扩展知识点

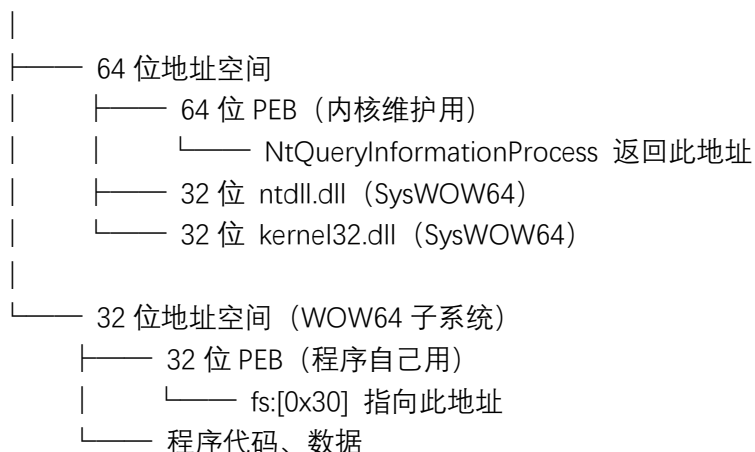
WOW64 机制的物理本质

场景：一个 32 位程序 `mushiai_chs.exe` 运行在 64 位 Windows 上。

- 系统为它在 `C:\Windows\SysWOW64\` 里准备了 32 位的 `ntdll.dll`、`kernel32.dll`。
- 它的 `fs:[0x30]` 指向 32 位 PEB（供 32 位代码自己使用）。
- 但同时，系统会在 64 位地址空间里，为这个 WOW64 进程额外维护一个 64 位 PEB（用来跟内核沟通）。
- `NtQueryInformationProcess(ProcessWow64Information)` 返回的非零值，就是那个 64 位 PEB 的地址。

图示：

64 位 Windows 内核



GetModuleHandleA 与 EnumProcessModules 对比

维度	GetModuleHandleA	EnumProcessModules
作用	直接问“某个模块在哪”	列出所有模块
返回	单个 HMODULE	多个 HMODULE 数组
效率	高（直接索引）	低（遍历链表）
适用范围	当前进程	任意进程（需 <code>OpenProcess</code> ）

GetProcAddress 的工作原理

```
pNtQueryInformationProcess NtQuery =
(pNtQueryInformationProcess)GetProcAddress(hNtdll, "NtQueryInformationProcess");
```

- 输入：模块基址 + 函数名字符串。
- 内部流程：
 - 定位 `IMAGE_EXPORT_DIRECTORY`。
 - 遍历 `AddressOfNames` 数组，用 `strcmp` 匹配函数名。
 - 通过 `AddressOfNameOrdinals` 找到索引。
 - 通过 `AddressOfFunctions[索引]` 得到函数 RVA。

加上模块基址，得到函数绝对地址。

- 失败返回 NULL：函数名拼写错误或模块未导出该函数。

为什么用 `GetModuleHandleA` 而不用 `GetModuleHandleW`

- 本课代码中模块名是 "ntdll.dll"（窄字符），所以用 A 版本。
- 若使用 L"ntdll.dll"（宽字符），则需用 W 版本。

两者功能完全一致，只是字符编码不同。

NtQueryInformationProcess 的其他常用查询类

枚举值	值	查询内容
ProcessBasicInformation	0	进程基本信息（PEB 地址、PID、PPID）
ProcessWow64Information	26	WOW64 状态（本课）
ProcessImageFileName	27	进程镜像文件名
ProcessDebugPort	7	调试端口（反调试）
ProcessDebugObjectHandle	30	调试对象句柄（反调试）

为什么 `OpenProcess` 需要 `PROCESS_VM_READ` 权限

```
HANDLE hProcess = OpenProcess(
    PROCESS_QUERY_INFORMATION | PROCESS_VM_READ,
    FALSE,
    pid
);
```

- `PROCESS_QUERY_INFORMATION`：允许查询进程信息（如 `NtQueryInformationProcess`）。
- `PROCESS_VM_READ`：允许读取进程内存（后续跨进程读取 PEB、模块时使用）。
- 注意：`PROCESS_QUERY_INFORMATION` 不包含读取内存权限，必须显式加上 `PROCESS_VM_READ`。

本课代码调用链映射到硬件

```
用户代码
|
|—— GetModuleHandleA("ntdll.dll")
|   |—— 访问当前进程 PEB → Ldr → 遍历模块名匹配 "ntdll.dll"
|
|—— GetProcAddress(hNtdll, "NtQueryInformationProcess")
|   |—— 解析 ntdll.dll 导出表 → 返回函数地址
|
|—— NtQuery(hProcess, ProcessWow64Information, &wow64Info, ...)
|   |
|   |—— 进入内核态 (syscall)
|   |
|   |—— 内核查询目标进程的 EPROCESS 结构
|       |—— 若为 64 位进程 → 返回 0
|       |—— 若为 32 位进程 → 返回 64 位 PEB 地址
|
|—— 读取 wow64Info → 判断位数
```

硬件层总结：

- 用户态调用 NtQueryInformationProcess → 通过 syscall 进入内核。
- 内核查询目标进程的 EPROCESS 结构，检查 Wow64Process 字段。
- 返回结果到用户态缓冲区。

必掌握的核心代码片段

- 获取 ntdll.dll 基址：HMODULE hNtdll = GetModuleHandleA("ntdll.dll");
- 动态获取函数地址：GetProcAddress(hNtdll, "NtQueryInformationProcess");
- 强转函数指针：(pNtQueryInformationProcess)GetProcAddress(...);
- 调用查询：NtQuery(hProcess, ProcessWow64Information, &wow64Info, sizeof(wow64Info), NULL);
- 判断结果：if (wow64Info == 0) { / 64 位 / } else { / 32 位 / }